

Instituto Tecnológico de Costa Rica

Curso:

TI-3600 Base de Datos

Profesor/a:

Jaqueline Solís Céspedes

Estudiante:

Andrés Bello Ramírez

Asignación:

Proyecto #1

Martes, 15 de setiembre

II Semestre, 2026

Administración de Tecnologías de Información

Alcance del proyecto

El Proyecto 1 consistió en la intervención de un sistema existente de agenda digital, desarrollado en Python con PostgreSQL como motor de base de datos. El sistema base administra usuarios, categorías jerárquicas, eventos y participaciones. La intervención agregó tres de los cuatro módulos disponibles: Disponibilidad de Usuarios y Gestión de Tiempos, Eventos Recurrentes, y Tareas Asociadas a Eventos. El módulo de Gestión de Ubicaciones no fue seleccionado.

Cada módulo se desarrolló en cuatro fases: modelo conceptual, modelo lógico, modelo físico con script SQL, e interfaz gráfica en Python. Cuando un requerimiento no generaba una entidad nueva, sino una consulta o un mecanismo de generación de datos, esa fase se omitió y el trabajo inició directamente en el modelo físico, según correspondiera.

Módulo de Disponibilidad de Usuarios y Gestión de Tiempos (RF-11, RF-12)

Modelo conceptual y lógico

Se definieron dos entidades: disponibilidades, que registra franjas horarias de un usuario con su fecha, hora de inicio, hora de fin y un tipo asociado, y tipos_disponibilidad, un catálogo con los valores Disponible, Ocupado y No disponible. En una primera versión, tipos_disponibilidad se intentó representar como un atributo multivaluado de disponibilidades. Se corrigió al identificar que un registro de disponibilidad solo puede tener un estado a la vez, y que el enunciado pedía explícitamente un catálogo separado.

Las cardinalidades quedaron establecidas así: usuarios (0,N) con disponibilidades (1,1), y tipos_disponibilidad (0,N) con disponibilidades (1,1). Un usuario puede no tener ninguna franja registrada sin que eso sea inválido, y un valor del catálogo puede existir sin que ninguna franja lo use todavía.

RF-12 no generó ninguna entidad nueva. El requerimiento se resolvió por completo mediante consultas SQL sobre las entidades ya existentes (eventos, participaciones y disponibilidades), ya que se trata de una capacidad de análisis y no de almacenamiento.

Modelo físico

Las tablas nuevas siguieron la convención de nombres ya usada en el sistema base: las llaves foráneas se nombran igual que la llave primaria que referencian (id_usuario, id_tipo_disponibilidad). Se identificó que el orden de creación de las tablas importa cuando existen dependencias por llave foránea: tipos_disponibilidad debe crearse antes que disponibilidades, porque esta última la referencia. Se agregó una restricción CHECK para garantizar que hora_fin sea mayor que hora_inicio en cada franja registrada.

Para RF-12 se construyeron tres subconsultas NOT EXISTS independientes. La primera descarta usuarios con un evento propio que choque con el rango consultado. La segunda hace lo mismo para eventos donde el usuario es invitado, a través de la tabla participaciones. La tercera descarta usuarios con una franja declarada como Ocupado o No disponible que choque con el rango consultado. Se estableció como regla de negocio que un usuario sin ninguna franja registrada para un rango se considera disponible por defecto, siempre que no tenga un evento que choque con ese rango.

Nota personal: para decidir si dos rangos de horario deben considerarse inclusivos o exclusivos en sus límites, comparé el comportamiento contra Microsoft Teams, donde una reunión puede iniciar en el mismo instante en que otra termina sin que eso se considere un traslape. Ese criterio se adoptó para toda comparación de horarios del proyecto.

Interfaz gráfica

Se agregó la pestaña Disponibilidad, con el CRUD de franjas siguiendo el mismo patrón visual que las pestañas existentes del sistema base, y una segunda sección dentro de la misma pestaña para consultar quién está libre en un rango de fecha y hora determinado, correspondiente a RF-12.

Hallazgos e incidencias

Durante las pruebas se detectó un error de decodificación de caracteres al leer texto con tildes desde la base de datos, a pesar de que esta tenía codificación UTF8 declarada a nivel general. La causa más probable fue una discrepancia entre la codificación de cliente usada al insertar cierto texto y la codificación esperada por la aplicación. Se corrigió forzando la codificación de cliente a UTF8 en cada conexión nueva, como práctica defensiva independiente del sistema donde corra la aplicación.

También se presentó un error de ejecución al integrar el código de la pestaña de Disponibilidad en el archivo principal. La causa fue un comentario ubicado en la columna cero del archivo, que interrumpió el nivel de indentación esperado y dejó los métodos nuevos fuera del alcance de la clase principal, funcionando como funciones sueltas en vez de métodos de la aplicación. Se corrigió reindentando el bloque completo y verificando, mediante análisis de la estructura del archivo, que los métodos quedaran correctamente asociados a la clase.

Módulo de Eventos Recurrentes (RF-13, RF-14)

Modelo conceptual y lógico

Se definió la entidad series_eventos como contenedora de la regla de periodicidad de una serie de eventos. Sus atributos son tipo_periodo (Días, Semanas, Meses o Años), cantidad_unidades_x_repeticion (cada cuántas unidades se repite la serie), fecha_inicio_serie y fecha_final_serie. Este último atributo se dejó opcional para representar series indefinidas, sin fecha de cierre conocida.

La relación entre series_eventos y eventos es de uno a muchos: una serie genera muchas ocurrencias, y cada ocurrencia pertenece como máximo a una serie. Se agregó la columna id_serie_evento a la tabla eventos como llave foránea opcional, ya que la mayoría de los eventos del sistema no pertenecen a ninguna serie.

Modelo físico

Para el atributo tipo_periodo se evaluaron dos alternativas de diseño: una tabla catálogo, siguiendo el mismo patrón que tipos_disponibilidad, o una restricción CHECK directa sobre la columna. Se optó por el CHECK, ya que la lista de valores es corta, cerrada, y no se espera que cambie con el tiempo.

Se decidió que, al eliminar una serie, sus ocurrencias generadas se eliminen también, ya que estas existen únicamente como consecuencia de la serie que las generó. Esto se implementó mediante ON DELETE CASCADE en la llave foránea de eventos hacia series_eventos.

RF-14 se resolvió como un algoritmo en la capa de aplicación, no como una entidad ni como una función dentro de la base de datos. Se construyeron funciones de suma de fechas que respetan la duración real de cada mes, usando únicamente la librería estándar de Python, sin dependencias externas nuevas, de forma que sumar un mes a una fecha como el 31 de enero resulte en un día válido del mes siguiente, ajustado automáticamente según corresponda.

Nota personal: decidí que el horizonte de generación para las series indefinidas fuera de seis meses a partir de la fecha de inicio, como punto intermedio entre no saturar la base de datos con eventos hacia un futuro lejano y mantener cobertura suficiente para el uso normal del sistema.

Interfaz gráfica

Se agregó la pestaña Eventos Recurrentes, con un formulario para definir una serie nueva (tipo de periodo, cantidad, fechas, y los datos del evento base), y un botón que genera todas las ocurrencias en una sola operación. La pantalla incluye un listado de las series existentes con el conteo de ocurrencias generadas, y un listado de las ocurrencias de la serie seleccionada. La generación de una serie se implementó como una única transacción de base de datos: si alguna ocurrencia fallara a la mitad del proceso, ninguna se guarda, evitando series incompletas.

Hallazgos e incidencias

En un primer intento se usó el tipo de dato TINYINT UNSIGNED para la cantidad de repetición, propio de MySQL y no disponible en PostgreSQL. Se corrigió con SMALLINT y una restricción CHECK que garantiza valores positivos.

Se intentó agregar la columna id_serie_evento a la tabla eventos como obligatoria (NOT NULL), lo cual falló porque la tabla ya contenía filas sin ningún valor posible para esa columna nueva. La corrección, dejarla opcional, también resultó ser la decisión correcta desde el punto de vista conceptual, ya que no todo evento pertenece a una serie.

Se presentó de forma repetida un error de relación inexistente al ejecutar sentencias SQL en sesiones nuevas de administración de la base de datos. La causa fue que el search_path, el esquema donde se buscan las tablas por defecto, se configura por sesión y no persiste entre pestañas de la herramienta de administración. Se resolvió de forma permanente fijando el search_path a nivel de toda la base de datos, de modo que cualquier conexión nueva lo herede automáticamente sin necesidad de configurarlo cada vez.

Módulo de Tareas Asociadas a Eventos (RF-15, RF-16, RF-17)

Modelo conceptual y lógico

Se definió la entidad tareas, vinculada a eventos y a usuarios. Sus atributos son título, descripción, prioridad, estado y fecha límite. Los valores de prioridad (Alta, Media, Baja) no estaban definidos en el enunciado y se establecieron como parte del diseño. Los valores de

estado (Pendiente, En progreso, Completada, Cancelada) sí estaban definidos explícitamente en el enunciado.

En un primer intento, la relación entre tareas y usuarios se describió como muchos a muchos, permitiendo más de un responsable por tarea. Se corrigió a uno a muchos, un usuario puede tener muchas tareas pero cada tarea tiene un único responsable, al revisar que el enunciado especifica un usuario responsable asignado, en singular.

Modelo físico

Ambas llaves foráneas (id_evento, id_usuario) se implementaron como columnas simples obligatorias, consistente con relaciones uno a muchos en los dos casos. Se agregaron restricciones CHECK para los valores cerrados de prioridad y de estado.

RF-16 y RF-17 se resolvieron con consultas de agregación agrupadas por usuario. Se construyó un reporte único que combina dos métricas mediante la cláusula COUNT con FILTER: la cantidad de tareas activas (Pendiente o En progreso) y la cantidad de tareas vencidas (fecha límite ya pasada y estado distinto de Completada o Cancelada). Se usó un LEFT JOIN desde usuarios hacia tareas para que los usuarios sin tareas asignadas también aparezcan en el reporte, con los conteos en cero.

Interfaz gráfica

Se agregó la pestaña Tareas, con el CRUD de tareas en la parte superior de la pantalla y el reporte de carga de trabajo en la parte inferior de la misma pestaña, actualizándose automáticamente con cada cambio registrado.

Hallazgos e incidencias

En la consulta de RF-16 se detectó una discrepancia de mayúsculas entre el texto usado en la condición de búsqueda (En Progreso) y el valor real almacenado según la restricción CHECK ya definida en la tabla (En progreso). PostgreSQL compara texto de forma sensible a mayúsculas, por lo que la consulta no arrojaba ningún error, simplemente omitía en silencio a los usuarios con tareas en ese estado. Este fue uno de los hallazgos más relevantes del proyecto: una consulta sintácticamente correcta puede producir resultados incompletos sin ninguna señal de error, razón por la cual se adoptó como práctica verificar cada consulta con datos reales antes de darla por válida.

Matriz de trazabilidad

Requerimiento	Descripción	Fases cubiertas	Artefacto principal	Estado
RF-11	Registro y gestión de disponibilidad temporal por usuario	Conceptual, Lógico, Físico, Interfaz	Tablas disponibilidades y tipos_disponibilidad; pestaña Disponibilidad	Completo
RF-12	Consulta de usuarios disponibles en un rango horario	Físico, Interfaz	Consulta con tres NOT EXISTS; sección de	Completo

Requerimiento	Descripción	Fases cubiertas	Artefacto principal	Estado
			búsqueda en pestaña Disponibilidad	
RF-13	Definición de series de eventos periódicos	Conceptual, Lógico, Físico, Interfaz	Tabla series_eventos; columna eventos.id_serie_evento; pestaña Eventos Recurrentes	Completo
RF-14	Generación automatizada de ocurrencias	Físico, Interfaz	Función generar_fechas_serie; generación transaccional desde la interfaz	Completo
RF-15	Seguimiento y control de tareas por evento	Conceptual, Lógico, Físico, Interfaz	Tabla tareas; pestaña Tareas	Completo
RF-16	Carga de trabajo y vencimientos	Físico	Consulta agrupada con COUNT y FILTER	Completo
RF-17	Reporte de carga de trabajo y tareas pendientes por usuario	Físico, Interfaz	Reporte integrado en pestaña Tareas	Completo

Pruebas y validación

Cada consulta y cada flujo de la interfaz se probó contra datos reales antes de considerarse terminado, incluyendo casos límite: fechas de reuniones consecutivas sin traslape, meses de distinta duración al sumar periodos, una tarea completada con fecha límite vencida que no debe contarse como pendiente, y usuarios sin ninguna tarea o franja registrada. Esta práctica permitió detectar, antes de la defensa, errores que no generan ningún mensaje visible pero sí resultados incorrectos, como el caso de sensibilidad a mayúsculas descrito en el módulo de Tareas.

Conclusión

Los tres módulos seleccionados, Disponibilidad de Usuarios y Gestión de Tiempos, Eventos Recurrentes, y Tareas Asociadas a Eventos, quedaron completos en sus fases correspondientes, con las consultas analíticas y los mecanismos de generación automática probados sobre datos reales. El diseño se mantuvo consistente con las convenciones ya establecidas en el sistema base, en particular la nomenclatura de llaves foráneas y el patrón visual de las pantallas existentes.

Declaraciones Uso de IA

Campo	Descripción
Ejercicio	Todo el documento.
Herramienta(s) utilizada(s)	Claude (Sonnet 5, high performance)
Etapas del trabajo	Formato de documento.
Descripción breve del uso	Como se especifica en el propio documento; este proyecto se desarrolló en iteraciones con una inteligencia artificial. Los datos almacenados en el chat con la IA, fueron aprovechados para la creación de este informe final, asegurando no solo los hallazgos, si no que cada mínimo detalle como errores, dudas y problemas que surgieron en mi proceso de aprendizaje para realizar los objetivos esperados. Este documento fue revisado y hecho, en gran parte, por mi persona, pero en su gran mayoría, sobre todo en formato y ortografía, fue “finiquitado” por una IA.
Declaración de autoría	Declaro que comprendo y puedo explicar la totalidad del trabajo entregado, y que la IA fue utilizada únicamente como apoyo, no como sustituto de mi trabajo.